LINUX SYSTEMS AND TROUBLESHOOTING

Week 1

Linux Foundations

TEXTBOOK & LAB MANUAL

IN THIS BOOK

Chapter	Title	Hands-on
1	Linux CLI Foundations	Concepts and orientation exercises
2	Files and Directories	Lab: linux-files-lab
3	Reading Files and Text Inspection	Lab: reading-files-lab
4	Search, Pipes, and Redirection	Lab: search-pipes-lab
5	Environment Variables, Command Lookup, and Shell Scripting	Lab: linux-path-script-lab

How to Use This Book

Each chapter teaches a topic and then walks you through it at the keyboard. Explanation and practice alternate, so read a section, then do its **Try it** steps before moving on.

Explanation and guided practice

TRY IT **Example: a guided exercise**

1. Numbered steps tell you what to do, in order.
2. Commands you type appear in teal-barred blocks:

```
ls -l notes.txt
```

3. Grey-barred blocks show output you should expect. Your exact output may differ:

```
-rw-r--r-- 1 timi timi 0 Oct  3 09:00 notes.txt
```

Pale blocks are diagrams and directory trees. They are for reading, not typing.

```
notes.txt
    |
    ▼
  cat
    |
    ▼
stdout
```

KEY CONCEPT

Teal callouts highlight an idea worth remembering.

WARNING

Amber callouts mark safety rules and common traps.

End-of-chapter work

REVIEW QUESTIONS

Check understanding in your own words.

PRACTICE EXERCISES

Short repeat-until-fluent tasks, done in the chapter's lab directory.

TAKE-HOME ASSIGNMENTS

Larger tasks to complete and hand in on your own time.

WORK SAFELY

Do all hands-on work inside the lab directories created in each chapter. Never practise destructive commands in /, /home, /etc, /var, or any directory that holds real files.

CHAPTER 1 · WEEK 1

Linux CLI Foundations

Concepts and orientation

What You Will Learn

By the end of this chapter you should be able to:

- ☐ Explain what Linux is and how the kernel differs from a distribution.
- ☐ Distinguish a terminal, a shell, a CLI, and a GUI.
- ☐ Describe the home directory and the current working directory.
- ☐ Tell absolute paths from relative paths.
- ☐ Name the main top-level directories and say what each one holds.

1.1 What Is Linux?

At its core, Linux is a Unix-like operating system. A computer is a stack of layers: **hardware** sits at the bottom, the **operating system** manages its resources in the middle, and **user-space applications** (shells, browsers, editors) sit at the top. Linux provides the foundational services that let hardware and user programs interact safely and efficiently.



1.2 Kernel vs. Distribution

People say "Linux" to mean two different things.

Term	Meaning
The kernel	The central core of the system, developed originally by Linus Torvalds and the Linux community. It manages CPU processes, memory, device drivers, and system calls, and talks directly to the hardware. On its own it is not a fully usable operating system.
A distribution (distro)	A complete, runnable operating system packaged around the kernel. A distribution such as Ubuntu, Debian, Fedora, or Red Hat bundles the kernel with GNU core utilities, a package manager, desktop environments, and ready-made applications.

REMEMBER

When someone says "I installed Linux", they installed a distribution. The kernel is one part of it.

1.3 Terminal, Shell, CLI, and GUI

These four terms are often mixed up. Each names a different thing.

Interfaces: GUI vs. CLI

A **Graphical User Interface (GUI)** provides visual components (windows, icons, menus, a pointer) built on display technologies such as the X Window System, GNOME, or KDE. A **Command Line Interface (CLI)** is a text-based interface where you type commands to instruct the operating system. This course lives mostly in the CLI because it is faster, scriptable, and precise.

Programs: Terminal vs. Shell

Program	Role
Terminal (terminal emulator)	A graphical window, such as gnome-terminal, konsole, or xterm. It draws the window, handles keyboard input, and renders text output.
Shell	The command interpreter running inside the terminal, such as bash or zsh. It prints a prompt (often ending in `$` for normal users), reads what you type, interprets it, starts a child process to run it, and returns the output.

```
You ──▶ Terminal window ──▶ Shell (bash) ──▶ Command (ls, grep, ...)
          draws text         interprets         does the work
```

TRY IT **meet your shell**

1. Open a terminal. The text before the cursor is the **prompt**; the shell is waiting for you.
2. Ask the system which shell is configured for your account:

```
echo $SHELL
```

3. You should see something like:

```
/bin/bash
```

> **WHY IT MATTERS**
> Almost everything in the next four chapters is something you ask the shell to do. Chapter 5 returns to the shell itself and shows that it is just another program.

1.4 Directories and Paths

Two locations to know

- **Home directory (`~`):** your personal workspace, such as `/home/username`, where your configuration files and documents live. A regular user can normally write only inside their home directory.
- **Current working directory (CWD):** the place in the filesystem where your shell session currently "stands". Commands that are given a bare name look here.

Absolute vs. relative paths

Kind	Starts at	Example
Absolute	The root, `/`	`/etc/passwd`, `/home/user/notes.md`
Relative	The current directory	`notes.md`, `projects/linux`, `../backups`

An absolute path always points to the same place, wherever you are. A relative path is a route from where you are now, using two special markers: `.` means the current directory and `..` means its parent.

TRY IT move around

1. Print the current directory:

```
pwd
```

2. List what is inside it:

```
ls
```

3. Change to a system directory using an absolute path, then confirm:

```
cd /etc
pwd
```

4. Go up one level with `..`, then jump straight home with `~`:

```
cd ..
pwd
cd ~
pwd
```

> **CHECKPOINT**
> After step 4, `pwd` should print your home directory. If it does not, run `cd ~` again.

1.5 The Filesystem Hierarchy

Linux organizes everything under a single root directory, `/`. There are no drive letters. The main top-level directories are:

Directory	Purpose
`/` (root)	The top-level starting point of the entire filesystem tree.
`/home`	Individual user home directories.
`/etc`	System-wide configuration files and administrative settings.
`/var`	Dynamic data such as system logs, spool files, and databases.
`/tmp`	Temporary files created by applications and users (often cleared on reboot).
`/usr`	User software binaries, libraries, and system documentation.
`/dev`	Special device files that represent hardware devices.
`/proc`	Virtual filesystem exposing real-time process statistics and kernel status.
`/sys`	Virtual filesystem giving structured access to hardware devices and kernel drivers.

TRY IT look at the top of the tree

1. List the root directory:

```
ls /
```

2. Compare it with the table above. Which directories do you recognise?

3. Peek at one of the virtual directories (read-only, nothing to break):

```
ls /proc | head -5
```

> **NOTE**
> You will meet `head` properly in Chapter 3 and the pipe (`|`) in Chapter 4. For now, treat the command as "show me the first few entries".

REVIEW QUESTIONS

Answer in your own words.

1. What is the difference between the Linux kernel and a Linux distribution?
2. What is the difference between a terminal and a shell?
3. You are in `/home/timi/projects`. Write the relative path to `/home/timi/backups`, and the absolute path.
4. Which top-level directory would you check first for a system log? For a configuration file?

PRACTICE EXERCISES

Do these in a terminal; no files are created.

1. Print your current directory, then use `cd` with only relative paths to reach `/usr/bin` and come back home.
2. Run `ls -l /` and find three directories from the table above. What do the first letters of each line tell you?
3. Open a second terminal window and run `pwd`. Do the two terminals share a current directory? Explain what that tells you about each window's shell.

CHAPTER 2 · WEEK 1

Files and Directories

Lecture and guided lab

What You Will Learn

By the end of this chapter you should be able to:

- ☐ Create directories with `mkdir` and whole directory trees with `mkdir -p`.
- ☐ Create files and update timestamps with `touch`.
- ☐ Explain how a path decides where something is created.
- ☐ Copy files and directories with `cp` and `cp -r`, and predict what happens when the destination already exists.
- ☐ Move and rename with `mv`, including with relative paths that climb the tree.
- ☐ Delete files and directories with `rm`, and use `-i`, `-r`, and `-f` appropriately.
- ☐ Explain how the shell expands wildcards such as `*`, and why `rm -rf` is dangerous.
- ☐ Predict what a command will do before you run it.

LAB SAFETY

Do all hands-on work in the dedicated lab directory created below. Never experiment with `rm -r`, `rm -rf`, or wildcards in `/`, `/home`, `/etc`, `/var`, or any directory that holds real files.

2.1 Setting Up Your Lab

Five commands manage almost everything you will do with files and directories:

Command	What it does
`mkdir`	Creates new directories.
`touch`	Creates empty files, or updates the timestamps of existing ones.
`cp`	Copies files and directories.
`mv`	Moves or renames files and directories.
`rm`	Removes (deletes) files and directories.

Graphical file managers let you drag and drop. Command-line utilities are faster, scriptable, and more precise, but they also do exactly what you type. Practising in a sandbox lets you make mistakes safely.

TRY IT create the sandbox

1. Find out where you are:

```
pwd
```

2. Create the lab directory, enter it, and confirm:

```
mkdir linux-files-lab
cd linux-files-lab
pwd
ls
```

3. `ls` prints nothing because the directory is empty. Every exercise in this chapter happens inside `linux-files-lab/`.

2.2 Creating Directories with mkdir

`mkdir <name>` creates one directory. You can list several names to create them all at once.

TRY IT make directories

1. Create one directory and check it exists:

```
mkdir documents
ls
```

```
documents
```

2. Create three at once:

```
mkdir downloads projects backups
```

3. Create a directory, then try to create it again:

```
mkdir notes
mkdir notes
```

4. The second command fails because `notes` already exists. Read the error message; it is telling you exactly what is wrong.

Nested directories and `-p`

5. Try to create a path whose parents do not exist yet:

```
mkdir projects/linux/week1
```

6. This fails. `projects` exists, but `projects/linux` does not, and plain `mkdir` will not invent missing parents. Add the `-p` (parents) flag:

```
mkdir -p projects/linux/week1
```

7. Inspect the result:

```
ls projects
ls projects/linux
```

> **THE RULE OF THUMB**
> `mkdir a/b/c` means: create `c`, assuming `a/b` already exists. `mkdir -p a/b/c` means: create whatever parts of this path do not already exist.

2.3 Creating Files with touch

`touch` has two jobs. If the file does not exist, it creates an empty one. If the file already exists, it leaves the contents alone and updates the timestamps to the current time.

TRY IT create a file and inspect it

1. Create a file and list it:

```
touch notes.txt
ls
```

```
notes.txt
```

2. Look at its details:

```
ls -l notes.txt
```

3. The size column shows `0` because the file is empty.

TRY IT watch the timestamp change

1. Create a file and note its time:

```
touch timestamp.txt
ls -l timestamp.txt
```

2. Wait a few seconds, then touch it again and compare:

```
touch timestamp.txt
ls -l timestamp.txt
```

3. The file is still empty and the contents were not overwritten. Only the modification time moved.

2.4 Paths Decide Where Things Happen

Every command that takes a file name takes a **path**. The path, not the command, determines where the result lands. A bare filename means "here, in the current directory".

TRY IT relative paths

1. You are inside `linux-files-lab/`. Create a file with a bare name:

```
touch file1.txt
```

This creates `linux-files-lab/file1.txt`.

2. Create a file inside an existing subdirectory by naming the path:

```
touch documents/file2.txt
```

```
linux-files-lab/
├── file1.txt
└── documents/
    └── file2.txt
```

Notice that `touch` did not create `documents`; it already existed. `touch` creates files, not parent directories.

3. Go several levels deep in one command and verify:

```
touch projects/linux/week1/lesson.txt
ls projects/linux/week1
```

TRY IT absolute vs. relative

1. Create one file in your current directory and one in `/tmp`, then look for each:

```
touch investigation.txt
touch /tmp/investigation.txt
ls investigation.txt
ls /tmp/investigation.txt
```

2. The two commands differ only in where the file lands:

```
touch investigation.txt        →  here (current directory)
touch /tmp/investigation.txt   →  / ── tmp/ ── investigation.txt
```

Suppose `pwd` prints `/home/timi/linux-files-lab`. Then `touch /home/timi/linux-files-lab/test.txt`, `touch test2.txt`, and `touch documents/test3.txt` all create files in the lab. The first uses an absolute path, the other two use relative ones.

> **KEY CONCEPT**
> A path determines where the file is created. A bare filename means "here".

2.5 Copying with cp

`cp source destination` duplicates a file. The original stays where it is.

TRY IT copy a file

1. Create a file with some content, then copy it:

```
touch original.txt
echo "This is the original file." > original.txt
cat original.txt
cp original.txt copy.txt
ls
```

```
original.txt
copy.txt
```

```
cat copy.txt
```

The content is identical.w

> **A NOTE ON >**
> The `>` in `echo ... > original.txt` sends the output of `echo` into the file instead of the screen. Chapter 4 explains redirection in full.

What if the destination already exists?

2. Change `copy.txt`, then copy over it:

```
echo "This is different." > copy.txt
cat copy.txt
cp original.txt copy.txt
cat copy.txt
```

3. The existing file was **silently overwritten** with the contents of `original.txt`. By default `cp` neither asks nor creates a name like `copy (1).txt`. That is why `cp` deserves caution.

4. The protective option is `-i` (interactive). It asks before overwriting:

```
cp -i original.txt copy.txt
```

5. A prompt such as `cp: overwrite 'copy.txt'?` appears and you must answer. Think of `-i` as "are you sure?". It is a safety check, not an undo: once you confirm, the old content is gone.

Copying several files

6. Create three files and a destination directory, then copy them all in one command:

```
touch file1.txt file2.txt file3.txt
mkdir archive
cp file1.txt file2.txt file3.txt archive/
ls archive
```

```
file1.txt
file2.txt
file3.txt
```

The general pattern is `cp source1 source2 source3 destination/`. The destination must already exist as a directory.

Copying directories needs `-r`

7. Try to copy a directory the plain way:

```
cp documents documents-backup
```

8. You get an error along the lines of `cp: -r not specified; omitting directory 'documents'`. A directory is a container that may hold files and more directories, so `cp` refuses to treat it like a single file. Add `-r` (recursive):

```
cp -r documents documents-backup
ls
```

```
documents
documents-backup
```

`-r` tells `cp`: copy this directory and descend through everything inside it.

2.6 Moving and Renaming with mv

`mv` does two jobs with one command, because both are the same operation underneath: changing where a directory entry points. Unlike `cp`, it does not leave the original behind.

You type	It means
`mv old.txt new.txt`	The target is a new name in the same place: **rename**.
`mv file.txt directory/`	The target is an existing directory: **move into it**, keeping the name.

TRY IT rename and move

1. Rename a file and check the old name is gone:

```
touch oldname.txt
mv oldname.txt newname.txt
ls
```

2. Move a file into a directory:

```
touch report.txt
mv report.txt documents/
ls documents
```

```
report.txt
```

TRY IT «mv» can overwrite

3. Create two files with different contents and move one onto the other:

```
echo "Version A" > fileA.txt
echo "Version B" > fileB.txt
mv fileA.txt fileB.txt
cat fileB.txt
```

4. "Version B" is gone. Like `cp`, `mv` replaces an existing destination without asking. Use `mv -i` to be prompted.

Worked example: moving up the tree

Suppose you are in `linux-files-lab/projects/linux/week2` and want to move `challenges.txt` into `linux-files-lab/documents/notes/`. A first attempt often looks like this:

```
mv challenges.txt documents/notes/
```

It fails, because `documents/notes/` is relative to your **current** directory and there is no `documents` inside `week2`. Look at where you are:

```
linux-files-lab/
├── documents/
│   └── notes/
└── projects/
    └── linux/
        └── week2/    ← YOU ARE HERE
```

To reach `linux-files-lab` you climb three levels (week2 → linux → projects → linux-files-lab), and each level up is `..`. So:

```
mv challenges.txt ../../../documents/notes/
ls ../../../documents/notes/
```

An absolute path works too and needs no counting: `mv challenges.txt ~/linux-files-lab/documents/notes/`.

MIND THE SPACES

Spaces separate arguments. `mv challenges .txt ~/linux-files-lab/documents/notes/` is read as two source files, `challenges` and `.txt`, which is why it fails with `cannot stat 'challenges'`. The filename is `challenges.txt`, with no space.

THREE QUESTIONS BEFORE ANY MV OR CP

Where am I? Where is the file? Where is the destination?

2.7 Removing with rm

`rm` permanently deletes files. There is no Trash and no undelete: the file system unlinks the entry and frees the data blocks immediately.

TRY IT remove files

1. Create and delete one file, then check it is gone:

```
touch temporary.txt
rm temporary.txt
ls temporary.txt
```

2. The last command reports that the file does not exist. Now try the interactive option, which asks first:

```
touch important.txt
rm -i important.txt
```

3. Remove several files in one command:

```
touch a.txt b.txt c.txt
rm a.txt b.txt c.txt
```

TRY IT remove directories

4. Plain `rm` does not remove directories:

```
rm archive
```

It fails because `archive` is a directory. For an **empty** directory, `rmdir` works. To remove a directory **and everything inside it**, use `-r`:

```
mkdir test-delete
touch test-delete/file1.txt test-delete/file2.txt
ls test-delete
rm -r test-delete
ls test-delete
```

5.The directory and everything in it are gone.

`-r` means the same thing in `cp` and `rm`

In both commands `-r` means **operate recursively on a directory and everything inside it**. The operation is different: `cp -r` recursively copies, `rm -r` recursively removes.

The dangerous combination

```
rm -rf directory
    │└─ f = force: do not ask, ignore errors
    └── r = recursive: go through every level
```

`rm -rf directory` removes the directory and its contents without asking for confirmation. Run on a parent directory, or with superuser privileges, it can permanently wipe important parts of a system.

> **RULE**
> Do not use `rm -rf` outside your controlled lab while you are learning, and avoid it with variables, wildcards, or system paths until you fully understand how they expand.

2.8 Wildcards and the Shell

Wildcards such as `*` are expanded by the **shell**, before the command runs. The command never sees the `*`; it sees the list of matching names.

TRY IT watch expansion happen

1.Make a disposable directory with some files:

```
mkdir wildcard-lab
cd wildcard-lab
touch one.txt two.txt three.txt one.html two.html notes.md
ls
```

```
notes.md  one.html  one.txt  three.txt  two.html  two.txt
```

2.List only the text files:

```
ls *.txt
```

The shell expanded `*.txt` into `one.txt three.txt two.txt` before `ls` ran.

3.Remove only the HTML files and check the rest survived:

```
rm *.html
ls
```

The dangerous typo

Compare the two commands below. They differ by a single space.

Command	What the shell actually sees
`rm *.html`	One pattern. It expands to the matching names: `rm one.html two.html`.
`rm * .html`	Two arguments: `*` and `.html`. The `*` expands to **every** file in the directory, followed by a file named `.html`.

So `rm * .html` can become `rm notes.md one.txt three.txt two.txt .html`. The `.html` part only produces an error, but the other files have already been passed to `rm`.

> **HABIT WORTH FORMING**
> Before running `rm` with a wildcard, run the same pattern with `ls` or `echo` first and read the list of names it expands to.

Chapter Summary

- A **path** decides where files are created, copied, moved, and deleted. A bare name means "here".
- `mkdir -p` builds missing parents; plain `mkdir` does not.
- `cp` and `mv` overwrite an existing destination silently; `-i` asks first.
- `-r` means "recursive" in both `cp` and `rm`; it is required for directories.
- `rm` is permanent. `rm -rf` removes recursively without asking.
- The shell expands `*`, spaces, `~`, and paths **before** the command runs.

```
Command
   ↓
Shell parses the command
   ↓
Shell expands things like *
   ↓
Command receives arguments
   ↓
Command operates on the filesystem
```

That chain is the beginning of systems thinking: you are learning to predict how the shell and the filesystem interact.

REVIEW QUESTIONS

Use your lab to investigate; do not answer from memory.

1. What is the difference between `touch file.txt` and `touch /tmp/file.txt`?
2. What happens if you run `cp source.txt destination.txt` and `destination.txt` already exists?
3. What is the difference between `cp directory backup` and `cp -r directory backup`?
4. What is the difference between `mv file.txt new.txt` and `mv file.txt documents/`?
5. Why does `rm directory` fail, even when the directory is empty?
6. What does `-r` mean in `cp -r` and `rm -r`? Is it conceptually the same?
7. What does `-i` protect you from, and what does it not protect you from?
8. Why is `rm *.txt` different from `rm * .txt`?t

PRACTICE EXERCISESba

Short tasks to repeat until the commands feel natural. Work inside `linux-files-lab/`.

1. Build this tree without pasting a prepared command sequence:

```
linux-files-lab/
├── backups/
│   └── week1/
├── documents/
│   ├── notes/
│   └── reports/
├── projects/
│   └── linux/
│       ├── week1/
│       └── week2/
└── scripts/
```

2. Create `documents/notes/linux.txt`, `documents/reports/lab-report.txt`, `projects/linux/week1/commands.txt`, `projects/linux/week2/challenges.txt`, and `scripts/backup.sh`. Put a line of text in each with `echo` and check them with `cat`.
3. Copy `documents/` into `backups/week1/` so that `backups/week1/documents/notes/linux.txt` exists. No GUI.
4. Rename `projects/linux/week1/commands.txt` to `linux-commands.txt`.
5. Move `projects/linux/week2/challenges.txt` into `documents/notes/`.
6. Create `scripts/test1.sh`, `scripts/test2.sh`, and `scripts/test3.sh`, then remove all three with a wildcard. Before you run it, write down exactly what the shell will expand the wildcard into.

TAKE-HOME ASSIGNMENTS

Hand in a short write-up. It is worth keeping in a GitHub systems-learning repository as evidence that you can operate, investigate, and document a system.

Assignment 2.1: Server project. You are given an empty directory called `server-config`. Create a structure for a hypothetical server project; populate it with placeholder configuration, log, backup, and script files; create a backup copy; rename one configuration file; move one file to another directory; and safely delete a temporary directory. Use only `pwd`, `ls`, `cd`, `mkdir`, `touch`, `cp`, `mv`, `rm`, `cat`, and `echo`. Then explain why every command you used did what it did.

Assignment 2.2: Lab notes. Create a short `lab-notes.md` containing:

- ☐ The directory tree you built.
- ☐ The commands you used and what each one did.
- ☐ Answers to the 8 review questions.
- ☐ Three mistakes or errors you encountered, and how you diagnosed and fixed each one.
- ☐ A short explanation of why `rm -rf` and wildcards require caution.

CHAPTER 3 · WEEK 1

Reading Files and Text Inspection

Lecture and guided lab

What You Will Learn

By the end of this chapter you should be able to:

- ☐ Read short files with `cat` and navigate large files with `less`.
- ☐ Search inside a file from within `less`.
- ☐ Read just the beginning of a file with `head` and just the end with `tail`.
- ☐ Monitor a file as it changes with `tail -f`.
- ☐ Explain the difference between reading a file and editing it.
- ☐ Choose the right tool for a file based on its size and your purpose.
- ☐ Describe how these tools fit into real troubleshooting.

> **THE GUIDING QUESTION**
> "How much of this file do I actually need to see?"

3.1 Inspecting Files Without Editing Them

In Linux, system configurations, program scripts, and runtime logs are mostly plain text files. Opening a large file in an editor such as `nano` or `vim` can use a lot of memory, because editors typically try to load the whole file at once. For simply looking, Linux provides four dedicated tools:

Tool	Purpose
`cat`	Concatenates files and dumps them directly to standard output.
`less`	Pages through a file interactively, one screenful at a time.
`head`	Shows the top portion of a file.
`tail`	Shows the bottom portion of a file.

These are inspection tools, not editors. Reading and editing are different jobs:

```
cat     → read/display
less    → read/navigate
head    → read the beginning
tail    → read the ending
nano    → edit
vim     → edit
```

TRY IT set up the lab

1. Create a directory for this chapter and enter it:

```
mkdir reading-files-lab
cd reading-files-lab
pwd
ls
```

2. The directory should be empty.

3.2 cat: Reading a Whole File

`cat` (short for *concatenate*) reads one or more files and writes their contents, in order, to standard output, which is your terminal.

```
notes.txt ──▶ cat ──▶ stdout ──▶ terminal
```

It is ideal for short files that fit on one screen, and for feeding text into other tools (you will do that in Chapter 4).

TRY IT create and read a small file

1. Build a three-line file:

```
echo "Linux is an operating system." > notes.txt
echo "The kernel manages system resources." >> notes.txt
echo "The shell provides a command interface." >> notes.txt
```

2. Read it:

```
cat notes.txt
```

```
Linux is an operating system.
The kernel manages system resources.
The shell provides a command interface.
```

Why the first line uses > and the others >>

`>` means "write, replacing what is there": it creates the file, or empties it first if it exists. `>>` means "append to the end" and keeps what is already there.

```
>   →  discard existing contents, write new contents
>>  →  existing contents + new contents
```

The first command establishes the file; the next two add to it. If you used `>` three times you would end up with only the last line. Both are **redirection operators handled by the shell**, not options of `echo`. Chapter 4 covers them properly.

TRY IT see where cat runs out of road

3. Generate a 100-line file with a loop (you will use this file throughout the chapter):

```
for i in {1..100}
do
    echo "Line $i: Linux systems administration practice." >> large-file.txt
done
ls -l large-file.txt
```

4. Now dump it:

```
cat large-file.txt
```

5. The whole file scrolls past and the beginning disappears above your screen. `cat` has no interactive navigation. That is its limitation, and the reason `less` exists.

Concatenation

6. Predict, then check, what this prints:

```
cat notes.txt large-file.txt
```

You should see all of `notes.txt`, then all of `large-file.txt`. Now try two small files in both orders:

```
echo "First file" > file1.txt
echo "Second file" > file2.txt
cat file1.txt file2.txt
cat file2.txt file1.txt
```

The output order follows the argument order. `cat` does not care that the inputs are separate files: it produces one continuous stream, *file1 contents + file2 contents*.

3.3 less: Reading Interactively

`less` is a **pager**. It shows one screenful at a time and does not load the whole file before starting, so it stays fast on very large files. It also lets you move backwards and search.

Key	Action
`Space` or `Page Down`	Advance forward by one screenful.
`b` or `Page Up`	Move backward by one screenful.
`↓` / `↑`	Scroll down/up by one line.
`G`	Jump to the end of the file.
`g` (or `1G`)	Jump to the beginning of the file.
`/pattern`	Search forward for a string.
`n`	Jump to the next match.
`q`	Quit and return to the shell prompt.

TRY IT drive less

1.Open the large file:

```
less large-file.txt
```

2.Practise each movement in turn:

- Press `Space`, then `Page Down`, then `↓`. Notice the difference between one screenful and one line.
- Press `b`, then `Page Up`, then `↑` to go backwards.
- Press `g` to jump to the beginning, then `G` to jump to the end. Capitalisation matters.

3.Search for a string: press `/`, type `Line 50`, press `Enter`. `less` jumps to the first match. Press `n` for the next one.

4.Press `q` to quit.

> **MEMORISE THIS ONE**
> `q` quits `less`. It is the key everyone forgets first.

TRY IT a real system file

5.Inspect a real configuration file. Do not modify anything:

```
less /etc/passwd
```

6.Press `g` then `G`, then search with `/root` and `Enter`. Quit with `q`.

7.Now compare it with:

```
cat /etc/passwd
```

`cat` says "show me everything immediately"; `less` says "let me inspect this interactively". The difference matters more and more as files get larger.

WHEN TO USE WHICH

Use `cat` for short files and for pipelines. Use `less` for large files (logs, code bases) and whenever you want to search or scroll back without cluttering your terminal history.

3.4 head and tail: Reading the Ends of a File

When you only need a slice, ask for just the slice. By default both commands show 10 lines. `-n N` chooses the number.

TRY IT head

1.Show the first 10 lines, then count what you saw:

```
head large-file.txt
head large-file.txt | wc -l
```

```
10
```

Do not worry about `wc` yet; Chapter 4 covers pipelines. For now: *head gives the first 10 lines, `wc -l` counts lines.*

2.Ask for a specific number:

```
head -n 5 large-file.txt
```

```
Line 1: Linux systems administration practice.
Line 2: Linux systems administration practice.
Line 3: Linux systems administration practice.
Line 4: Linux systems administration practice.
Line 5: Linux systems administration practice.
```

3.Think of `head -n N file` as "give me the first N lines".

A realistic use of head

Imagine a 50,000-line configuration export. You do not need all of it. Start with `head -n 30 configuration.txt` and ask: what kind of file is this, what format does it use, are there headers, does it look corrupted, does the structure look as expected? This is a very common administrative first step.

TRY IT tail

4.Show the last lines:

```
tail large-file.txt
tail -n 5 large-file.txt
```

You should see Lines 96 to 100.

```
head → beginning
tail → end
```

Why the end matters

For a log such as `application.log`, you usually care about what happened **recently**. `cat application.log` floods the screen with old history; `tail application.log` immediately shows the newest entries.

3.5 tail -f: Watching a File Change

`tail -f` (**f**ollow) is the first tool in this course that behaves like real monitoring. It prints the end of the file and then **stays running**, streaming new lines as they are appended.

TRY IT watch a log live

You need two terminal windows or tabs.

1. In **Terminal 1**, create an empty log and follow it. The cursor will appear to hang. That is intentional: it is waiting for new data.

```
touch application.log
tail -f application.log
```

2. In **Terminal 2**, move to the same directory and append messages one at a time:

```
cd ~/reading-files-lab
echo "User logged in" >> application.log
echo "Database connection established" >> application.log
echo "ERROR: Connection timeout" >> application.log
```

3. After each command, look at Terminal 1. The new line appears immediately.

4. In Terminal 1, press `Ctrl + C` to stop following and return to the prompt.



THE PATTERN

One process writes data while another process monitors it. You will meet this constantly with application logs, web servers, databases, system services, authentication logs, and monitoring systems.

3.6 Choosing the Right Tool

Match the tool to the question you are asking.

Situation	Tool
A 5-line configuration file.	`cat`
A 200,000-line log you want to browse and search for one error.	`less`
See how a configuration file begins.	`head`
See the latest entries in a log.	`tail`
Watch new log messages while you reproduce a problem.	`tail -f logfile`

Building a realistic practice log

Let us create a log that looks like the real thing, using a **here-document**:

```
cat > server.log << 'EOF'
2026-10-03 09:00:01 INFO Server starting
2026-10-03 09:00:03 INFO Network interface initialized
2026-10-03 09:00:05 INFO SSH service started
2026-10-03 09:01:10 INFO User timi logged in
2026-10-03 09:02:15 INFO Backup started
2026-10-03 09:03:22 INFO Backup completed
2026-10-03 09:04:30 WARNING Disk usage above 75%
2026-10-03 09:05:11 INFO User timi logged out
2026-10-03 09:06:45 ERROR Database connection failed
2026-10-03 09:07:02 INFO Database connection restored
2026-10-03 09:08:10 INFO Backup started
2026-10-03 09:09:15 INFO Backup completed
EOF
```

The first line is the interesting one. Read it in three pieces:

Piece	Meaning
`cat`	With no file argument, `cat` reads from the keyboard and prints what it receives.
`> server.log`	Send `cat`'s output into `server.log` (overwriting it if it exists).
`<< 'EOF'`	A **here-document**: keep accepting input until a line containing only `EOF`, and give all of it to `cat`.

So the whole command means: *create or overwrite `server.log` with this block of text.* `EOF` is just a marker; any word works (for example `END`), as long as the closing marker matches the opening one. The quotes around `'EOF'` tell the shell to treat the text literally, with no variable expansion. With a bare `EOF`, a line such as `Hello $name` would have `$name` replaced by its value; with `'EOF'` it stays `Hello $name`.

TRY IT investigate the log like an administrator

You are told: *"The application is behaving strangely. Check the server log."* Do **not** start by opening the whole file.

1. See the shape of the file, then what happened most recently:

```
head server.log
tail server.log
```

2. Now investigate in detail:

```
less server.log
```

3. Inside `less`, search for `ERROR` (press `/`, type `ERROR`, `Enter`), then for `WARNING`. Quit with `q`.

This is the working pattern for real logs:

```
Large log
   │
   ▼
tail / head  ──  understand its shape
   │
   ▼
less         ──  search and inspect
   │
   ▼
tail -f      ──  watch live while you reproduce the problem
```

TRY IT a mini incident

Terminal 1:

```
touch live.log
tail -f live.log
```

Terminal 2:

```
echo "INFO: Server started" >> live.log
sleep 2
echo "INFO: Network initialized" >> live.log
sleep 2
echo "WARNING: High memory usage" >> live.log
sleep 2
echo "ERROR: Database timeout" >> live.log
sleep 2
echo "INFO: Database recovered" >> live.log
```

Watch Terminal 1; you have just simulated a basic real-time logging workflow. Press `Ctrl + C` when done.

Chapter Summary

- `cat` prints everything; `less` lets you navigate and search; `head` and `tail` show the ends; `tail -f` follows a growing file.
- Choose by asking how much of the file you actually need to see.
- A troubleshooting workflow: look at the **shape** (head/tail), **investigate** (less), then **monitor** (tail -f).
- Later you will combine these with `grep`, `sort`, `uniq`, `wc`, `cut`, `awk`, and `sed`. For example:

```
tail -n 100 application.log | grep ERROR
```

That is no longer just reading a file; it is extracting the information you care about. The progression is **Read** → **Search** → **Filter** → **Transform** → **Analyze**.

REVIEW QUESTIONS

Without your notes, explain in your own words:

1. Why would you use `cat config.txt` instead of `less config.txt`?
2. Why is `less` useful for large files?
3. What is the difference between `head file` and `tail file`?
4. What does `tail -f` actually do?
5. Why is `tail -f` particularly useful for system administration?
6. What happens when you press `q` inside `less`?
7. What happens when you press `Ctrl+C` while `tail -f` is running?
8. With a 500,000-line log and interest only in the most recent 20 lines, which command do you use?
9. To search interactively through that log, which command do you use?
10. To watch the log continuously while reproducing a problem, which command do you use?

PRACTICE EXERCISES

Short tasks to repeat in `reading-files-lab/`.

1. Without using `cat`, show lines 1 to 3 of `server.log`, then its last 2 lines.
2. Open `/etc/passwd` in `less`, jump to the end, jump back to the beginning, and search for your own username.
3. Use `head` and `wc -l` together to prove that `head -n 20 large-file.txt` prints exactly 20 lines.
4. Explain why `cat > server.log << 'EOF'` overwrites the file but `cat >> server.log << 'EOF'` would not.

TAKE-HOME ASSIGNMENTS

Complete and keep your commands in a notes file.

Assignment 3.1: Build and inspect a mini server. Create:

```
reading-files-lab/
├── configs/
│   ├── server.conf
│   └── network.conf
├── logs/
│   ├── application.log
│   └── system.log
└── reports/
    └── status.txt
```

Populate the files so that `server.conf` has 5 lines, `network.conf` 10, `application.log` 50, `system.log` 100, and `status.txt` 3. Then record the command you used for each task:

1. Display `server.conf` completely.
2. Display only the first 3 lines of `network.conf`.
3. Display only the last 5 lines of `application.log`.
4. Open `system.log` in an interactive viewer and go to the beginning, the end, and a specific search result.
5. Monitor `application.log` in real time while appending new entries from another terminal.

Assignment 3.2: Diagnose without cat. Imagine `application.log` has 100,000 lines and the problem happened recently. You must see the latest entries, search the file, and watch for new entries while you reproduce the problem. Design the workflow using only `head`, `tail`, and `less`, and explain why each tool is the right choice at each step.

CHAPTER 4 · WEEK 1

Search, Pipes, and Redirection

Lecture and guided lab

What You Will Learn

By the end of this chapter you should be able to:

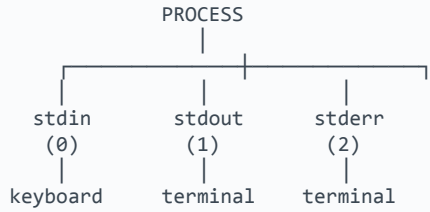
- ☐ Explain `stdin`, `stdout`, and `stderr` and file descriptors 0, 1, and 2.
- ☐ Redirect output with `>` and `>>`, input with `<`, and errors with `2>`.
- ☐ Redirect both output and errors, and explain why the order matters.
- ☐ Connect commands with the pipe `|`.
- ☐ Search file contents with `grep` and directory trees with `find`.
- ☐ Combine `find`, `grep`, `sort`, `wc`, `head`, and `tail` into small workflows.
- ☐ Predict what a pipeline will do before running it.

This chapter is a step up. You are moving from "how do I inspect a file?" to "how do I make Linux commands work together to answer a question?"

4.1 The Three Standard Streams

A central Unix idea is to build small, single-purpose tools that cooperate. They cooperate through standard streams, which every process receives automatically:

Stream	File descriptor	Default
stdin (standard input)	0	The keyboard.
stdout (standard output)	1	The terminal screen.
stderr (standard error)	2	The terminal screen, but a **separate** channel for errors and diagnostics.



The shell can **rewire** these connections without changing the programs themselves. Send stdout to a file with `command > file.txt`; connect one program's stdout to another's stdin with `commandA | commandB`.

TRY IT ordinary output

1. Set up this chapter's lab:

```
mkdir -p ~/search-pipes-lab
cd ~/search-pipes-lab
```

2. Run two commands and notice where their output goes:

```
echo "Hello Linux"
ls
```

```
echo ──▶ stdout ──▶ terminal
```

The terminal is simply the default destination for standard output.

4.2 Redirecting Output: > and >>

TRY IT send output to a file

1. Redirect `echo`'s output into a file instead of the screen:

```
echo "Hello Linux" > message.txt
cat message.txt
```

```
Hello Linux
```

Nothing was printed when you ran `echo`: the shell redirected stdout into `message.txt`. If the file does not exist, `>` creates it.

The dangerous part of >

2. Write to the same file twice:

```
echo "First version" > test.txt
cat test.txt
echo "Second version" > test.txt
cat test.txt
```

```
Second version
```

The first contents were replaced.

> **RULE**
> `>` means create or overwrite. This is one of the most important shell rules to internalise.

Appending with >>

3. Start fresh, then append twice:

```
echo "First line" > test.txt
echo "Second line" >> test.txt
echo "Third line" >> test.txt
cat test.txt
```

```
First line
Second line
Third line
```

```
>   → replace
>>  → append
```

> **PREDICT FIRST**
> Without running anything, what will `cat test.txt` show after these five commands? `echo "A" > test.txt`, `echo "B" >> test.txt`, `echo "C" >> test.txt`, `echo "D" > test.txt`, `echo "E" >> test.txt`. **Answer:** `D` then `E`, because the fourth command's `>` destroyed the earlier contents.

4.3 Redirecting Input: <

`command < file` connects the file to the command's stdin.

TRY IT input redirection

1.Create a five-line file and count its lines two ways:

```
printf "one\ntwo\nthree\nfour\nfive\n" > numbers.txt
wc -l < numbers.txt
```

```
5
```

```
wc -l numbers.txt
```

```
5 numbers.txt
```

2.Both count the lines, but they are different mechanisms:

Command	What happens
`wc -l numbers.txt`	`wc` receives the **filename** as an argument and opens the file itself.
`wc -l < numbers.txt`	The **shell** opens the file and feeds its contents to `wc` through stdin.

The difference matters later, with pipelines and programs that read stdin.

4.4 Redirecting Errors: 2>, 2>&1, and &>

Errors travel on stderr, not stdout. That is why `>` alone does not capture them.

TRY IT separate the two streams

1.Make a command fail on purpose:

```
ls does-not-exist.txt
```

2.That error went to stderr. Capture it into a file:

```
ls does-not-exist.txt 2> error.log
cat error.log
```

```
ls ──┬── stdout ──▶ terminal
     └── stderr ──▶ error.log
```

3.Now ask `ls` for one file that exists and one that does not, redirecting only stdout:

```
ls numbers.txt does-not-exist.txt > output.log
cat output.log
```

You find the success there, but the error **still appeared on screen**: `>` redirects stdout only.

4.Send stdout and stderr to **different** files:

```
ls numbers.txt does-not-exist.txt > success.log 2> failure.log
cat success.log
cat failure.log
```

This is useful in scripts and automation, where normal output and failures need different treatment.

5.Send both to the **same** file, two ways:

```
ls numbers.txt does-not-exist.txt > combined.log 2>&1
ls numbers.txt does-not-exist.txt &> combined2.log
cat combined.log
```

Why the order matters

Read `> combined.log 2>&1` left to right: first send stdout to `combined.log`, then send stderr to **wherever stdout is currently going**. Both end up in the same file. Written the other way round, stderr would be pointed at the terminal before stdout moved. `&>` is the Bash shorthand for the same thing.

Operator	Effect
`>`	stdout → file (overwrite)
`>>`	stdout → file (append)
`<`	file → stdin
`2>`	stderr → file
`2>&1`	stderr → wherever stdout is going
`&>`	stdout + stderr → file

> **GOOD TO KNOW**
> You rarely need to type the `1` in `1>`. A plain `> file` is shorthand for `1> file`.

4.5 Pipelines: Connecting Commands

A **pipe** (`|`) connects the stdout of one command directly to the stdin of the next. The data stays in memory; no temporary file is written, and the commands run concurrently.

```
command A ──stdout──▶ | ──stdin──▶ command B ──▶ terminal
```

TRY IT your first pipelines

1. Create a file with repeated words, then sort it through a pipe:

```
printf "apple\nbanana\napple\norange\nbanana\napple\n" > fruits.txt
cat fruits.txt | sort
```

2. Add a third stage that removes adjacent duplicates:

```
cat fruits.txt | sort | uniq
```

```
apple
banana
orange
```

Each program does one small job; the pipe composes them. That is the Unix philosophy.

> **PHILOSOPHY**
> Small tools that do one thing well can be composed into larger workflows.

Do not pipe needlessly

`cat fruits.txt | sort` works, but `sort fruits.txt` is simpler, because `sort` can read the file itself. Understand the pipeline, but do not add one merely because you can. Pipes earn their place when data is passing between **different processing stages**.

4.6 Searching Text with grep

`grep` (Global Regular Expression Print) scans files, or a stream on stdin, for lines that match a pattern and prints those lines.

Flag	Effect
`-i`	Ignore case.
`-v`	Invert: print lines that do **not** match.
`-n`	Prefix each match with its line number.
`-c`	Print only the count of matching lines.
`-l`	Print only the names of files that contain a match.

TRY IT search a log

1. Create a small log with a here-document (see Chapter 3 for how it works):

```
cat > server.log << 'EOF'
INFO Server started
INFO Network initialized
INFO User timi logged in
WARNING Disk usage high
INFO Backup started
ERROR Database connection failed
INFO Database connection restored
ERROR Authentication failure
INFO Backup completed
EOF
```

2. Find the errors:

```
grep "ERROR" server.log
```

```
ERROR Database connection failed
ERROR Authentication failure
```

3. Case matters by default, so this finds nothing; add `-i` and it matches:

```
grep "error" server.log
grep -i "error" server.log
```

4. Show where each error is, count them, and invert the match:

```
grep -n "ERROR" server.log
```

```
6:ERROR Database connection failed
8:ERROR Authentication failure
```

```
grep -c "ERROR" server.log
```

```
2
```

```
grep -v "ERROR" server.log
```

Asking `grep` directly beats opening the file in `less` and hunting by eye: the computer does the searching.

TRY IT grep inside pipelines

5. Count the INFO entries, then show only the last error:

```
grep "INFO" server.log | wc -l
grep "ERROR" server.log | tail -n 1
```

```
server.log ──▶ grep INFO ──▶ matching lines ──▶ wc -l ──▶ count
```

The second command reads as: *find all errors, then keep only the last one.* It is a very practical pattern.

4.7 Searching the Filesystem with find

`grep` searches **inside** files. `find` searches the filesystem **itself**, by name, type, size, or age.

```
grep → contents
find → filesystem
```

Test	Effect
`-name "pattern"`	Match file names with a string or wildcard. Use `-iname` to ignore case.
`-type f` / `-type d`	Regular files / directories only.
`-size <s>`	Filter by size.
`-mtime <days>`	Filter by modification age.

TRY IT explore a workspace

1.Build a small tree:

```
mkdir -p workspace/documents workspace/scripts workspace/archive workspace/logs
touch workspace/documents/report1.txt workspace/documents/report2.txt
touch workspace/archive/old_notes.txt
touch workspace/scripts/build.sh workspace/scripts/deploy.sh
touch workspace/logs/server.log workspace/logs/application.log
```

2.See everything beneath `workspace`:

```
find workspace
```

3.Find by name. Note that `find` examines **names and paths**, not file contents:

```
find workspace -name "*.txt"
```

```
workspace/documents/report1.txt
workspace/documents/report2.txt
workspace/archive/old_notes.txt
```

4.Narrow by type, then combine criteria. Read left to right: regular files only, whose names end in `.txt`:

```
find workspace -type f
find workspace -type d
find workspace -type f -name "*.txt"
```

5.Count and save the result:

```
find workspace -type f -name "*.txt" | wc -l
find workspace -type f -name "*.txt" > text_files_report.txt
cat text_files_report.txt
```

```
filesystem search ──▶ text stream ──▶ file report
```

Note that `find ... > report` needs no `cat`: `find` produces the output and the shell redirects it.

find vs. the shell's wildcard

6.Search the logs for errors using the shell's wildcard:

```
grep "ERROR" workspace/logs/*.log
```

The **shell** expanded `*.log` into the matching filenames before `grep` ran. `find` does its own recursive searching; a wildcard does not. Keep the two apart.

The three search questions

Question	Tool
Where is the file?	`find`
Which files have this type or name?	`find`
What does this file contain?	`grep`

4.8 Putting It Together: A Log Investigation

TRY IT from raw log to report

1. Create a log with 100 ordinary events and four notable ones:

```
for i in {1..100}
do
    echo "INFO Event $i completed successfully" >> workspace/logs/application.log
done
echo "ERROR Database connection failed" >> workspace/logs/application.log
echo "INFO Database connection restored" >> workspace/logs/application.log
echo "WARNING Disk usage above threshold" >> workspace/logs/application.log
echo "ERROR Authentication service unavailable" >> workspace/logs/application.log
```

2. Investigate in steps:

```
tail -n 10 workspace/logs/application.log
grep "ERROR" workspace/logs/application.log
grep -n "ERROR" workspace/logs/application.log
grep -c "ERROR" workspace/logs/application.log
```

3. Build a multi-stage pipeline and save its result:

```
grep "ERROR" workspace/logs/application.log | tail -n 2
grep "ERROR" workspace/logs/application.log | tail -n 2 > error_report.txt
cat error_report.txt
```

```
application.log ──▶ grep ──▶ ERROR lines ──▶ tail ──▶ last 2 errors ──▶ error_report.txt
```

The redirection happens **after the whole pipeline**, so the file holds the result of `tail`, not all the errors.

4. Capture a failure on stderr. Give `find` a path that does not exist, then redirect its error:

```
find workspace-does-not-exist -type f
find workspace-does-not-exist -type f 2> find-errors.log
cat find-errors.log
```

5. Separate success from failure in one command, then combine them:

```
ls workspace workspace-does-not-exist > success.log 2> failure.log
ls workspace workspace-does-not-exist > combined.log 2>&1
```

You now combine file reading, searching, pipelines, filtering, and redirection in a single workflow.

Chapter Summary

- Every process has stdin (0), stdout (1), and stderr (2); the shell can rewire them.
- `>` overwrites, `>>` appends, `<` feeds input, `2>` captures errors, `2>&1` and `&>` merge streams.
- `|` connects stdout to stdin. Build pipelines from small tools, and only where they add something.
- `grep` searches contents; `find` searches the filesystem; the shell expands wildcards before either runs.

```
Read → Search → Filter → Connect → Redirect → Automate
```

REVIEW QUESTIONS

1. What are file descriptors 0, 1, and 2, and where does each go by default?
2. Why does `ls missing.txt > out.log` still print an error on screen?
3. What is the difference between `wc -l file` and `wc -l < file`?
4. Why is the order in `> combined.log 2>&1` important?
5. When is `grep "x" file` better than `cat file | grep "x"`?
6. In one sentence each, when do you reach for `grep` and when for `find`?

PRACTICE EXERCISES

Repeat these in `search-pipes-lab/` until you can do them without looking.

1. Using only `find` and `wc -l`, answer: how many regular files, `.txt` files, `.log` files, and `.sh` files exist in `workspace`?
2. Create `workspace/reports/` and write the last 5 errors from the application log into `workspace/reports/error-report.txt`.
3. Predict, then run, each command and explain what it does:

```
echo "hello" > file.txt
echo "world" >> file.txt
grep -i "error" server.log | tail -n 5
find . -type f -name "*.txt" | wc -l
find . -type f -name "*.log" > logs.txt
ls /does-not-exist 2> error.log
cat file.txt | grep "hello"
```

4. The last command can be simplified. How?

TAKE-HOME ASSIGNMENTS

Document your answers in a markdown file.

Assignment 4.1: Three useful pipelines. Write three pipelines that solve real tasks. For each, give: what you wanted, the command, the input, the output, and why the pipeline works. Models:

	Pipeline 1
What I wanted	Count the regular `.txt` files inside the workspace.
Command	`find . -type f -name "*.txt" | wc -l`
Input	The file structure of the current directory.
Output	An integer, such as `12`.
Why it works	`find` outputs a stream of matching paths; the pipe feeds them to `wc -l`, which counts the lines.

	Pipeline 2
What I wanted	List users whose shell is bash, sorted, saved to a file.
Command	`grep "/bin/bash" /etc/passwd | sort > bash_users.txt`
Input	The file `/etc/passwd`.
Output	A sorted file, `bash_users.txt`.
Why it works	`grep` keeps the matching lines; `sort` orders them; `>` saves the stream.

	Pipeline 3
What I wanted	Keep the 20 most recent error lines from a log and add them to a daily report.
Command	`grep -i "error" /var/log/syslog | tail -n 20 >> daily_report.log`
Input	The system log `/var/log/syslog`.
Output	Entries appended to `daily_report.log`.
Why it works	`grep -i` extracts error lines in any case; `tail -n 20` keeps the latest 20; `>>` appends without destroying earlier reports.

Assignment 4.2: Challenge execution. Find all files of one extension inside a practice directory and produce a text report of their names without a GUI. Verify the report with `cat`.

Assignment 4.3: Server investigation. Create `server-lab/` with `configs/` (`server.conf`, `network.conf`), `logs/` (`application.log`, `system.log`), `scripts/` (`backup.sh`, `deploy.sh`), and an empty `reports/`. Populate the files, then answer each task **with commands**, not by reading the files:

1. Find all regular files. 2) Find all `.conf` files. 3) Find all `.log` files. 4) Count the `.log` files. 5) Search `application.log` for errors. 6) Search again ignoring case. 7) Show errors with line numbers. 8) Count the errors. 9) Show the last 5 errors. 10) Save those 5 to `reports/latest-errors.txt`. 11) Redirect the errors of an intentionally failing command into `reports/command-errors.log`. 12) Build one report listing all `.txt` files, all `.log` files, and all shell scripts using `find` and redirection.

CHAPTER 5 · WEEK 1

Environment Variables, Command Lookup, and Shell Scripting

Lecture and guided lab

What You Will Learn

By the end of this chapter you should be able to:

- ☐ Inspect and use environment variables, and explain quoting with `$`.
- ☐ Explain how `$PATH` controls command lookup and why its order matters.
- ☐ Distinguish builtins, external executables, aliases, and functions.
- ☐ Use `type`, `which`, and `man` to investigate commands.
- ☐ Explain why `hello-lab` fails while `./hello-lab` works.
- ☐ Explain that the shell is itself a program, and tell `$SHELL`, `$0`, and the running process apart.
- ☐ Read and interpret exit statuses with `$?`.
- ☐ Write, run, and troubleshoot a Bash script, and describe how a script becomes a process.

5.1 Environment Variables

Environment variables are key-value pairs maintained by the shell. They describe the context of your session and pass configuration to programs. To read one, put a dollar sign in front of its name.

TRY IT look at your environment

1. Set up this chapter's lab:

```
mkdir -p ~/linux-path-script-lab/scripts
cd ~/linux-path-script-lab
pwd
```

2. Print four common variables:

```
echo $HOME
echo $USER
echo $SHELL
echo $PATH
```

Almost all of these values can be different on another Linux machine.

Quotes decide whether the shell expands a variable

3. Predict what each of these prints, then run them:

```
echo $USER
echo "$USER"
echo '$USER'
```

```
timi
timi
$USER
```

Double quotes (and no quotes) let the shell **expand** `$USER` into its value. Single quotes make the shell treat it literally.

> **KEY CONCEPT**
> The shell performs expansion before the command receives its arguments.

5.2 $PATH and Command Lookup

When you type a command such as `ls` or `grep`, the shell does not search the whole filesystem. It consults `$PATH`, a colon-separated list of directories, and scans them **from left to right** for an executable with that name. The first match runs. If none matches, you see `command not found`.

TRY IT read your PATH

1. Print it one directory per line:

```
echo "$PATH" | tr ':' '\n'
```

You have combined variable expansion, `echo`, a pipe, and `tr`. A typical result:

```
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
/sbin
/bin
/usr/games
/usr/local/games
/snap/bin
```

2. Questions to answer from your own output: which directories hold system commands, which is searched first, and what happens if the same command exists in two of them? (A directory may even appear twice, as `/snap/bin` often does; that is redundant but harmless.)

3. See which files provide a command. `which` shows the first match and `type -a` shows every match Bash knows about:

```
which ls
type -a ls
```

Conceptually the shell walks the list and stops at the first hit:

```
ls
 ↓  /usr/local/sbin/ls ?   no
 ↓  /usr/local/bin/ls ?    no
 ↓  /usr/sbin/ls ?         no
 ↓  /usr/bin/ls ?          YES  → execute /usr/bin/ls
```

TRY IT prove that order matters

4. Create two directories, each holding a different script called `hello`:

```
mkdir -p ~/path-test/first ~/path-test/second
echo 'echo "I am FIRST"' > ~/path-test/first/hello
echo 'echo "I am SECOND"' > ~/path-test/second/hello
chmod +x ~/path-test/first/hello ~/path-test/second/hello
```

5. Save your real PATH first, so you can restore it, then put both directories at the front:

```
OLD_PATH="$PATH"
export PATH="$HOME/path-test/first:$HOME/path-test/second:$PATH"
hello
```

```
I am FIRST
```

6. Reverse the order and run it again:

```
export PATH="$HOME/path-test/second:$HOME/path-test/first:$PATH"
hello
```

```
I am SECOND
```

That is **PATH precedence**: the earlier directory wins. It is why changing `$PATH` can have real consequences. If `/usr/local/bin/python` and `/usr/bin/python` both exist, whichever directory comes first in `$PATH` decides which one runs.

7. Restore your original PATH:

```
export PATH="$OLD_PATH"
echo "$PATH"
```

> **TEMPORARY ONLY**
> Changing `$PATH` this way lasts for the current terminal session only. Do not edit `.bashrc` or `.profile` for these exercises.

> **A CAUTION ABOUT THE SIMPLE MODEL**
> "The shell scans PATH left to right" is a useful model, not the whole story. Bash also checks aliases, functions, and builtins first, and it **remembers** (hashes) where it found commands it has already run.

5.3 What Kind of Command Is It?

Not every command is a file on disk. Linux commands fall into four categories:

Type	Description
Shell builtin	Built into the shell itself, for example `cd`, `echo`, `exit`, `pwd`.
External executable	A compiled binary or script in a filesystem directory, for example `/usr/bin/ls`, `/bin/grep`.
Alias	A user-defined shortcut that expands into another command.
Function	A mini-script defined in the shell's memory.

TRY IT ask the shell how it reads a command

1. Run `type` on a mix of commands:

```
type ls
type cd
type pwd
type grep
type echo
```

Typical output; yours may differ:

```
ls is aliased to `ls --color=auto'
cd is a shell builtin
pwd is a shell builtin
grep is /usr/bin/grep
echo is a shell builtin
```

2. Compare with `which`, which only looks for executable files in `$PATH`:

```
type cd
which cd
type ls
which ls
```

`type` reports a builtin; `which cd` usually finds nothing. `type` answers "how does my current shell interpret this?", while `which` answers "which executable would I find through `$PATH`?".

3. Fill in this table in your notes for `cd`, `ls`, `grep`, `find`, and `echo`:

Command	type says	which finds
cd	?	?
ls	?	?
grep	?	?
find	?	?
echo	?	?

> **KEY LESSON**
> Do not equate "command" with "executable file". A command can be a builtin, an executable, an alias, a function, or a keyword. The shell resolves what you typed before execution.

Why cd must be a builtin

An external program runs in its **own process**, separate from the shell. If `cd` were external:

```
Terminal shell
    │
    └── launches cd process
            │
            ├── changes ITS OWN directory
            └── exits
→ the parent shell is still where it started
```

The child would change its own working directory and exit, leaving your shell in the same place. Because `cd` must change the current directory of **your** shell, the shell has to do it internally.

4. See the consequence:

```
pwd
cd /tmp
pwd
```

TRY IT read the manual

5. `man` opens the manual page for a command. Do not read it top to bottom; practise finding things:

```
man df
```

Press `/`, type `human`, press `Enter`, then `n` for the next match, and `q` to quit.

5.4 Running Your Own Programs: Names vs. Paths

TRY IT the file that "isn't there"

1. Create a tiny executable script:

```
cat > hello-lab <<'EOF'
#!/bin/bash
echo "Hello from my Linux lab!"
EOF
chmod +x hello-lab
ls
```

2. The file is right there. Now run it two ways:

```
hello-lab
```

```
bash: hello-lab: command not found
```

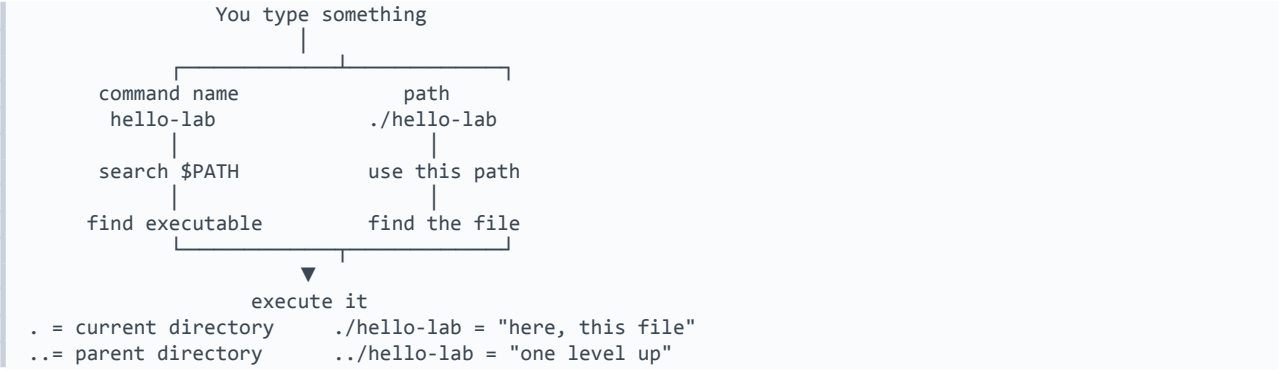
```
./hello-lab
```

```
Hello from my Linux lab!
```

Why one works and the other does not

`hello-lab` is a **command name**. For an external command, the shell searches `$PATH` for it. Your current directory is **not** in `$PATH`, so the search fails.

`./hello-lab` is a **path**. `.` means the current directory, so you are telling the shell exactly where the file is, and no search is needed.



Why isn't the current directory in $PATH?

Partly for security. If it were searched first, a malicious file named `ls` in a directory you entered would run when you typed `ls` expecting the real command. Linux makes you say explicitly, with `./`, that you mean the file here.

TRY IT **make the name work**

3. Temporarily add the lab directory to the **front** of `$PATH`, then run it by name:

```
OLD_PATH="$PATH"
export PATH="$PWD:$PATH"
echo "$PATH"
hello-lab
type hello-lab
```

4. It works, and `type` identifies your script. So the problem was never "Linux can't execute files in the current directory"; it was "the current directory was not in the command search path". Restore your PATH:

```
export PATH="$OLD_PATH"
hello-lab
./hello-lab
```

The name fails again; the explicit path still works.

KEY IDEA

`$PATH` is primarily a command **discovery** mechanism. `./` says: "Do not search. I am telling you exactly where it is."

TWO SEPARATE QUESTIONS

Can the shell **find** the file? (PATH or an explicit path.) And can Linux **execute** it? (The execute permission, set with `chmod +x`.) A found file without `x` gives `Permission denied`.

5.5 The Shell Is a Program Too

When you open a terminal, a **Bash process** is running. Bash is itself an executable file on disk; the operating system loaded it into memory and created a process from it.

```
/bin/bash          = executable file on disk
a running bash     = process, an instance of that program in memory
```

Programs are files too. Under `/usr/bin` sit `bash`, `grep`, `find`, `ls`, `cp`, and the rest. `/bin` is often a symbolic link to `/usr/bin` on modern Ubuntu, so `/bin/bash` and `/usr/bin/bash` can be the same file.

TRY IT find the Bash program

1.Locate it three ways:

```
which bash
type bash
ls -l "$(which bash)"
ls -l /bin/bash
```

2.Compare with the shebang in your scripts, `#!/bin/bash`. The script is explicitly asking for that file.

TRY IT start a shell from a shell

3.Run Bash by path and look at the shell level:

```
echo $SHLVL
/bin/bash
echo $SHLVL
exit
```

The level goes up inside the inner shell and returns when you `exit`:

```
Bash process #1
    └── runs /bin/bash
            └── Bash process #2
```

$SHELL, $0, and the real process

Three commands that sound alike answer different questions:

Command	What it tells you
`echo $SHELL`	Your **login shell**: the shell configured for your account. It is a variable, not a live check.
`echo $0`	The name the current shell was **invoked** as. A leading `-` (as in `-bash`) usually means a login shell.
`ps -p $$ -o comm=`	The **actual process** running as your shell right now. `$$` is the shell's process ID.

4.Run all three:

```
echo $SHELL
echo $0
ps -p $$ -o comm=
```

If you start another shell, for example `zsh`, `$SHELL` may still say `/bin/bash` while `$0` and the process say `zsh`: configuration, invocation name, and running process are three different things.

5.6 Exit Status

Every command returns an integer to the shell when it finishes, called its **exit status**. The special variable `$?` holds the status of the **most recently executed** command. By convention, `0` means success and anything from `1` to `255` means failure.

TRY IT read some exit statuses

1.Successful and failing commands:

```
true
echo $?
```

```
0
```

```
false
echo $?
```

```
1
```

```
ls /does/not/exist
echo $?
```

```
2
```

2.A subtle point: run these three lines together.

```
false
echo $?
echo $?
```

The first `echo $?` reports the status of `false`. The second reports the status of the **first echo**, which succeeded. `$?` is not a permanent record of the last failure; it always describes the command that just ran.

5.7 Your First Shell Script

A **shell script** is an executable text file containing commands that the shell reads and runs line by line. Three ingredients matter:

Part	Purpose
Shebang (`#!/bin/bash`)	On line 1, column 1. Tells the kernel which interpreter should run the rest of the file.
Comments (`#`)	Lines starting with `#` are ignored by the shell and serve as documentation.
Commands	Ordinary commands, executed top to bottom.

New text files are **not** executable by default. To run a script by path, set the execute permission with `chmod +x`.

TRY IT write and run the system summary script

1.Open an editor:

```
nano scripts/system-summary.sh
```

2.Enter this script, then save and exit nano:

```
#!/bin/bash

# Print section header
echo "==================================="
echo "       SYSTEM SUMMARY REPORT       "
echo "==================================="

# Print system hostname
echo "Hostname:"
hostname

# Print current system date and time
echo -e "\nCurrent Date and Time:"
date

# Print system uptime and load averages
echo -e "\nSystem Uptime:"
uptime

# Print summary of disk space usage
echo -e "\nDisk Usage:"
df -h /

echo "==================================="
```

3. Check it, then try to run it **before** making it executable:

```
cat scripts/system-summary.sh
./scripts/system-summary.sh
```

```
bash: ./scripts/system-summary.sh: Permission denied
```

4. Look at the permissions. The missing `x` is the problem:

```
ls -l scripts/system-summary.sh
```

```
-rw-r--r-- 1 timi timi ... system-summary.sh
```

5. Add execute permission and run it:

```
chmod +x scripts/system-summary.sh
ls -l scripts/system-summary.sh
```

```
-rwxr-xr-x
```

```
./scripts/system-summary.sh
```

What each line does

Line	Explanation
`#!/bin/bash`	The shebang: run this script with the Bash interpreter at `/bin/bash`.
`echo "====...===="`	Prints a decorative divider to standard output.
`echo "  SYSTEM SUMMARY REPORT  "`	Prints the main heading.
`echo "Hostname:"`	Prints a label for the next piece of information.
`hostname`	Prints the machine's host name.
`echo -e "\nCurrent Date and Time:"`	`-e` enables backslash escapes, so `\n` adds a blank line before the heading.
`date`	Prints the current date, time, and timezone.
`echo -e "\nSystem Uptime:"`	A heading preceded by a blank line.
`uptime`	Shows how long the system has been running, active users, and load averages.

Line	Explanation
`echo -e "\nDisk Usage:"`	A heading preceded by a blank line.
`df -h /`	Disk free for the root filesystem; `-h` prints human-readable units (MB, GB).

How a script becomes a process

When you type `./scripts/system-summary.sh`, this happens:

```
You type the command
      ↓
Bash sees an explicit path, so no PATH lookup
      ↓
The filesystem locates the file
      ↓
Execute permission is checked
      ↓
The kernel reads the shebang (#!/bin/bash)
      ↓
A new /bin/bash process is started for the script
      ↓
Commands run one after another; output goes to stdout
```

If you typed `system-summary.sh` with no path, the shell would search `$PATH` instead and report `command not found` unless the script's directory were listed there.

5.8 Scripts, Failure, and Redirection

TRY IT what happens when a line fails

1. Make a broken copy and add a bad command near the bottom (before the final `echo` line):

```
cp scripts/system-summary.sh scripts/system-summary-broken.sh
nano scripts/system-summary-broken.sh
```

```
this_command_does_not_exist
```

2. Make it executable and run it:

```
chmod +x scripts/system-summary-broken.sh
./scripts/system-summary-broken.sh
```

You see an error, **and the script carries on**: the final `echo` still runs. By default Bash does not stop a script just because a command failed.

3. Check the exit status of the whole script:

```
./scripts/system-summary-broken.sh
echo $?
```

The status of a script is generally the status of the **last command executed**, not of the first failure. So these three commands can give a script status of `0` even though the first failed:

```
command A → fails
command B → succeeds
command C → succeeds
```

That is why production scripts use constructs such as `set -e` and deliberate error handling. Do not add `set -e` yet; first learn the default behaviour.

TRY IT use what you learned about redirection

4. Save the report, then append a second one:

```
./scripts/system-summary.sh > system-report.txt
cat system-report.txt
./scripts/system-summary.sh >> system-report.txt
cat system-report.txt
```

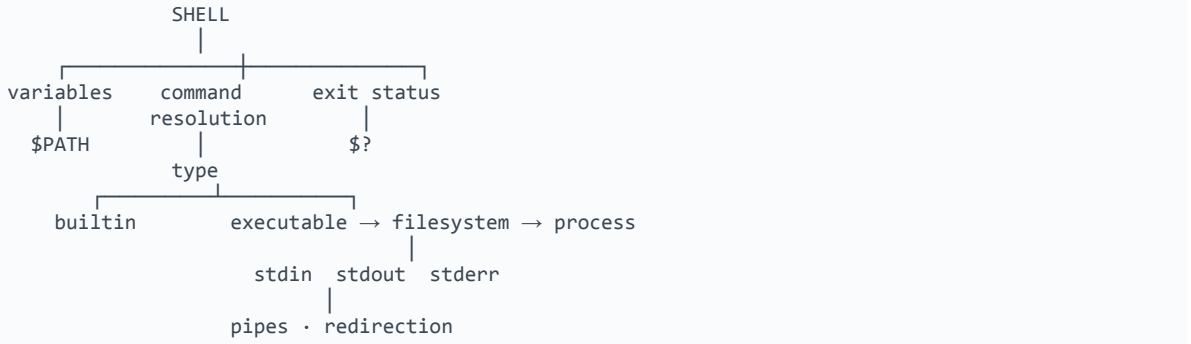
5. Capture the broken script's errors, then both streams:

```
./scripts/system-summary-broken.sh 2> script-errors.txt
cat script-errors.txt
./scripts/system-summary-broken.sh > output.txt 2>&1
cat output.txt
```

The ordering matters: `> output.txt` first redirects stdout, then `2>&1` sends stderr to wherever stdout is currently going.

Chapter Summary

- **Variables** such as `$HOME`, `$USER`, and `$PATH` hold session settings; the shell expands them before the command runs.
- **`$PATH`** is an ordered list of directories searched for external commands; earlier entries win. `./name` skips the search.
- A **command** can be a builtin, an executable, an alias, or a function. `type` tells you which.
- The **shell itself** is an executable file; a running shell is a process.
- **`$?`** holds the exit status of the last command: `0` means success.
- A **script** is a text file with a shebang and execute permission. By default it keeps going after a failure.



The shell is an execution environment, not just a place to type commands.

REVIEW QUESTIONS

Answer before looking anything up.

1. Why does `./hello-lab` work when `hello-lab` does not?
2. What would happen if you ran `chmod -x scripts/system-summary.sh` and then `./scripts/system-summary.sh`?
3. Why does `which cd` usually produce no result while `type cd` identifies it?
4. What is the difference between `echo $HOME` and `echo '$HOME'`?
5. If `$PATH` is `/usr/local/bin:/usr/bin:/bin` and `foo` exists, executable, in both `/usr/local/bin` and `/usr/bin`, which runs, and why?

6. What does `echo $?` actually represent?
7. If a script runs `command_A`, `command_B`, `command_C` and `command_B` fails, does Bash skip `command_C`?
8. What is the difference between `> report.txt` and `>> report.txt`?
9. What do `echo $SHELL`, `echo $0`, and `ps -p $$ -o comm=` each tell you?

PRACTICE EXERCISES

Short tasks for `linux-path-script-lab/`.

1. For `ls`, `cd`, `grep`, `find`, `echo`, and `hostname`, write down what `type` says and what `which` finds.
2. Run `man date` and use `/` to find how to print only the year. Quit with `q`.
3. Using two temporary scripts of your own, show that reordering `$PATH` changes which one runs, then restore the original PATH.
4. Run your system summary script with `>`, then again with `>>`, and explain the difference in the file.

TAKE-HOME ASSIGNMENTS

Submit the script and your investigation notes.

Assignment 5.1: System diagnostic script. Extend `system-summary.sh` into `scripts/system-diagnostic.sh`. It should print, in this layout:

```
========================================
        SYSTEM DIAGNOSTIC REPORT
========================================

Hostname:

Current User:

Home Directory:

Shell:

Current Directory:

Date and Time:

Uptime:

Disk Usage:

PATH:

========================================
```

Obtain every value with real commands or environment variables (`$USER`, `$HOME`, `$SHELL`, `$PATH`, `pwd`, `hostname`, `date`, `uptime`, `df`); do not hard-code your own values. Make it executable, run it, then save the output:

```
chmod +x scripts/system-diagnostic.sh
./scripts/system-diagnostic.sh
./scripts/system-diagnostic.sh > diagnostic-report.txt
```

Assignment 5.2: Investigation notes. Include answers to the review questions, and your lab directory should look approximately like this (temporary investigation files may differ):

```
linux-path-script-lab/
├── hello-lab
├── system-report.txt
├── diagnostic-report.txt
├── script-errors.txt
├── scripts/
│   ├── system-summary.sh
│   ├── system-summary-broken.sh
│   └── system-diagnostic.sh
```

The important deliverable is the working diagnostic script plus your notes.